

Computer Science and Engineering Department

NATIONAL INSTITUTE OF TECHNOLOGY DELHI



Session (2025–2026)

Big Data Analytics CSBB 422

‘IoT Sensor Data Analysis in Real Time’

Submitted To:

Dr. Priten
(Assistant Professor)
Computer Science
Engineering Department

Submitted By:

Abdul Arsh	221210002
Aditya Shaurya Singh Negi	221210012
Akshat Singh	221210015
Gautham Binoy	221210039
Immanuel Thomas Francis	221210050

B.Tech CSE, 7th Semester

Abstract

This project implements a containerized streaming pipeline to ingest, buffer, and persist IoT sensor data in real time. Sensor events from an Android device are published via MQTT, bridged into Apache Kafka, and periodically written to HDFS using date-partitioned storage. A lightweight Flask frontend renders near-real-time charts for accelerometer and gyroscope axes. This document summarizes architecture, deployment, commands, and usage.

Acknowledgment. We gratefully acknowledge the guidance and support provided by the Computer Science and Engineering Department, National Institute of Technology Delhi. Special thanks to **Dr. Priten** (Assistant Professor) for his insightful feedback, encouragement, and constructive suggestions throughout the development of this IoT sensor data analysis project.

Contents

1	Project Overview	2
2	Architecture	4
2.1	Logical Data Flow	4
2.2	Infrastructure Components	6
3	Prerequisites	7
4	Files and Repository Layout	10
5	Docker Compose Configuration	11
5.1	Key Settings	11
5.2	Sample Service Definitions (Excerpt)	11
5.3	Extended Compose Commands	12
5.4	Health and Logs	13
5.5	Resource Tuning (Inline Notes)	13
5.6	Updating Services Safely	13
5.7	Cleanup and Maintenance	14
5.8	Why Compose Here	14
6	Start / Stop Commands	15
7	Verifying Services & Web UIs	16
7.1	Kafka Topic Health	16
8	Python Clients (Source Code Overview)	17
8.1	m.py (MQTT → Kafka)	17
8.2	k.py (Kafka → HDFS)	17
8.3	Running Producers and Consumers	17
8.3.1	Illustrative Snippet	17
9	Frontend Web UI – IoT Sensor Visualizer	19
9.1	Summary	19
9.2	Endpoint Sketch	19
9.3	HDFS Integration Notes	19
9.4	Launching the UI	19

1 Project Overview

We developed a compact streaming pipeline for IoT telemetry focused on reliability, low latency, and modularity. The principal components are:

- **Ingestion:** Android device publishes sensor readings (e.g., accelerometer, gyroscope) at configurable sampling intervals to a public MQTT broker (`broker.emqx.io`) using lightweight JSON payloads.
- **Streaming Bridge:** `m.py` subscribes to MQTT topics, enriches messages (ingestion timestamp / device ID), and forwards them to Kafka topic `sensors`, decoupling edge capture from downstream consumers.
- **Storage:** `k.py` reads from Kafka, buffers records (time or count threshold), and writes JSON batches into HDFS under date partitions to reduce small-file proliferation and enable time-sliced analytics.
- **Visualizer:** Flask-based frontend queries recent HDFS data, exposing REST endpoints consumed by Chart.js to render motion traces (x, y, z axes) with manual and auto-refresh.

Storage Schema

Files are stored under:

```
/iot/sensors/date=YYYY-MM-DD/data.json
```

This supports simple time-series partitioning by date, enabling:

- Fast retrieval of a specific day's telemetry.
- Easy lifecycle management (delete old partitions).
- Straightforward integration with Spark/Hive (date predicate pushdown).

Representative Message Format

```
{
  "device_id": "and01",
  "sensor": "accelerometer",
  "timestamp": 1732540210,
  "x": -0.12,
  "y": 9.81,
  "z": 0.03
}
```

Design Highlights (Brief)

- **Loose Coupling:** MQTT → Kafka bridge isolates mobile protocol from internal persistence.

- **Scalability Path:** Increase Kafka partitions or add more consumer instances of `k.py`.
- **Observability:** UI provides immediate visual validation of ingestion integrity.
- **Extensibility:** Future sensors (temperature, magnetometer) can reuse topic structure with minimal code changes.

Potential Extensions

- Add anomaly detection (e.g., sudden acceleration spikes) processed in near-real-time.
- Introduce compression (Snappy / Gzip) for large JSON batches in HDFS.
- Migrate batch writes to Parquet for columnar efficiency and schema evolution.

2 Architecture

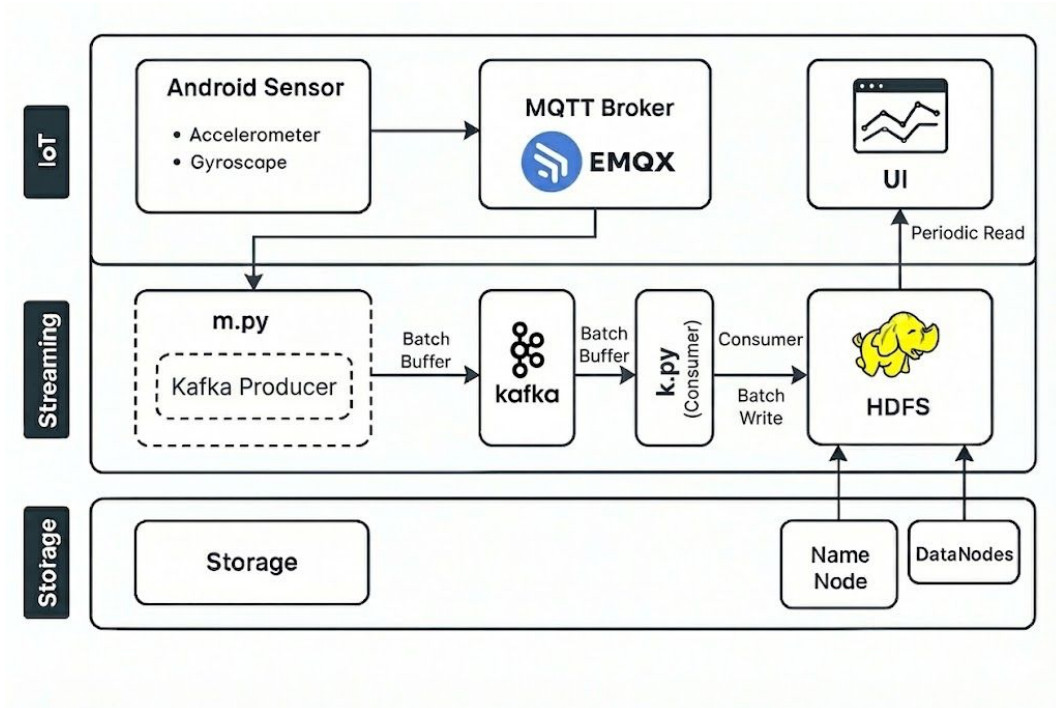


Figure 2.1: Overall system architecture: sensor publishing via MQTT, bridging into Kafka, batch persistence to HDFS, and visualization through Flask/Chart.js.

2.1 Logical Data Flow

Android Sensor → MQTT Broker → m.py (Python) → Kafka → k.py (Python) → HDFS
(NameNode + DataNodes)

The pipeline components illustrated in Figure 2.1 emphasize decoupling between ingestion (MQTT), streaming durability (Kafka), and long-term storage (HDFS), with a visualization layer for rapid feedback.

1. Android Sensor The mobile device's onboard sensors (accelerometer, gyroscope, etc.) produce time-stamped readings. A lightweight client application (or script) formats data as small JSON payloads and publishes them to predefined MQTT topics (e.g., `device/imu`).

2. MQTT Broker The public MQTT broker acts as the first aggregation point. It receives published messages and holds them transiently for subscribed clients. Using MQTT here reduces latency and simplifies intermittent connectivity handling on the mobile side because the broker buffers messages when clients are temporarily offline.

3. m.py (MQTT → Kafka Bridge) The `m.py` script runs as a continuously subscribed MQTT client. For each incoming message:

1. Parse JSON payload (validate fields: timestamp, x, y, z, sensor type).
2. Optionally enrich (add device ID, ingestion timestamp).
3. Produce the record asynchronously to Kafka topic `sensors`.

This decouples the IoT edge protocol (MQTT) from the internal streaming backbone (Kafka) and provides persistence guarantees beyond transient MQTT delivery.

4. Kafka Kafka stores records in a partitioned, append-only log. Offsets provide ordering guarantees per partition. Retention policies can be tuned (e.g., keep 3 days of raw sensor events). This layer enables scalable fan-out: multiple downstream consumers (analytics jobs, monitoring alerts) could read the same topic independently without interfering with ingestion.

5. k.py (Kafka → HDFS Consumer) The `k.py` script acts as a batch-oriented micro-ingestion process:

1. Poll Kafka for new messages (using a consumer group for offset management).
2. Buffer messages in memory until a batch threshold (count or time interval) is met.
3. Serialize the batch as NDJSON or JSON array and write to a date-partitioned path in HDFS using WebHDFS.
4. Flush and commit consumer offsets ensuring idempotent, exactly-once (or at-least-once) semantics depending on configuration.

Batching reduces small-file proliferation and amortizes network overhead.

6. HDFS (NameNode + DataNodes) The NameNode manages file metadata and namespace; DataNodes store blocks. Written batch files (e.g., `/iot/sensors/date=2025-11-25/data001.json`) become durable inputs for downstream analytics engines (Spark, Hive, etc.). Partitioning by date accelerates time-sliced queries and simplifies retention policies.

Optional Downstream Step: Visualization / Analytics A Flask/Chart.js frontend periodically reads the most recent partition and renders sensor traces, allowing real-time quality assurance and anomaly detection.

Data Integrity and Reliability

- **At Source:** MQTT QoS can be tuned (e.g., QoS 1 for at-least-once delivery).
- **Streaming Backbone:** Kafka persists messages until consumed; offsets enable replay.
- **Storage Layer:** HDFS replication safeguards against single DataNode failure.

Latency Considerations End-to-end latency is dominated by:

1. MQTT publish interval (sensor sampling frequency).
2. Kafka producer flush (often sub-millisecond for small payloads).
3. Batch interval in `k.py` (trade-off between latency and file efficiency).

Adjusting batch size and flush frequency can shift the pipeline between real-time and near-real-time modes.

Scalability Horizontal scaling occurs by:

- Increasing Kafka partitions for parallel consumers.

- Adding more consumer instances of `k.py` under the same group to distribute load.
- Expanding HDFS DataNodes to handle larger volumes.

Fault Tolerance If `m.py` fails, MQTT retains messages until it reconnects. If `k.py` fails mid-batch, uncommitted offsets allow safe replay upon restart, minimizing data loss risk. HDFS replication mitigates storage node failures.

2.2 Infrastructure Components

- **Hadoop:** A lightweight Hadoop Distributed File System (HDFS) setup using BDE Docker images. The cluster consists of one NameNode (metadata and namespace manager) and two DataNodes (block storage). This configuration is sufficient for demonstrating replication, partitioned directory layout, and WebHDFS access without heavy resource overhead. Default replication factor may be set to 1 or 2 (adjustable in `hdfs-site.xml`) depending on experimentation needs.
- **Kafka:** Apache Kafka (via Confluent Platform containers) provides a durable, high-throughput commit log for decoupling ingestion from persistence. Zookeeper (legacy coordination component) maintains broker metadata and topic partition assignments. The `sensors` topic can be configured with multiple partitions to enable parallel consumption and scale-out processing.
- **Networking:** A custom Docker bridge network (`bigdata`) supplies internal DNS-based hostname resolution (e.g., `kafka`, `namenode`, `datanode1`). This isolates traffic from the host network, avoids port clashes, and allows services to reference each other by container name rather than hard-coded IPs. All containers join this network at creation time through `docker-compose.yml`.
- **Python Client:** A dedicated container running Python 3.10 executes producer and consumer scripts. It installs runtime libraries (`paho-mqtt`, `confluent-kafka`, `hdfs`) and communicates outward to the MQTT broker (public) while internally addressing Kafka and HDFS endpoints. Using WebHDFS abstracts file writes to HDFS via HTTP, avoiding direct Java client complexity and enabling simpler REST-style operations.
- **MQTT Broker (External):** The public broker (`broker.emqx.io`) receives lightweight sensor messages from the mobile device. MQTT's publish/subscribe semantics minimize bandwidth and battery usage, making it suitable for mobile and IoT telemetry.
- **Flask Visualizer (Optional Service):** A containerized Flask application queries recent HDFS partitions and exposes JSON endpoints consumed by a Chart.js frontend. This provides near-real-time rendering of motion sensor axes to validate ingestion and storage correctness.
- **Time Partitioned Storage:** Directory scheme `/iot/sensors/date=YYYY-MM-DD/` enables efficient retrieval of daily batches and lays groundwork for downstream processing (e.g., Spark jobs or incremental analytics) while keeping metadata simple.

3 Prerequisites

System and Tooling Requirements

- **Host OS:** Windows 10/11 (with WSL2 recommended) or any modern Linux distribution (Ubuntu 22.04+, Fedora, etc.). macOS (Apple Silicon or Intel) also works if Docker Desktop is installed. Ensure virtualization is enabled in BIOS/firmware.
- **Container Engine:** Docker Desktop (Windows/macOS) or Docker Engine + Docker Compose plugin (Linux). Minimum version: Docker Engine 24.x; Compose plugin v2.20+ for stability with network aliases.
- **CPU/RAM:** At least 2 vCPUs and 4 GB RAM usable by Docker; recommended 4 vCPUs and 8 GB RAM when running Kafka + HDFS + UI concurrently. Adjust in Docker Desktop resources panel if containers are being OOM-killed.
- **Disk Space:** 2–3 GB free for base images (Kafka, Hadoop, Python). Additional space scales with ingested sensor data; consider pruning old images periodically.
- **Git (Optional but Recommended):** For cloning and version tracking (`git clone` workflow). Alternative: download ZIP archive from repository.
- **Python:** Not required on host (all execution uses `python:3.10` container). Host Python may be used for quick prototyping but is intentionally decoupled from pipeline runtime.
- **Network Connectivity:** Stable outbound internet connection (pull images, connect to public MQTT broker). If behind a proxy, configure Docker daemon JSON or environment variables accordingly.
- **User Permissions:** On Linux, membership in `docker` group avoids `sudo` (run `sudo usermod -aG docker <username>` then re-login). On Windows/macOS, Docker Desktop handles privilege escalation.
- **Time Synchronization:** System clock should be accurate; sensor timestamps rely on device + host time correlation for ordering.
- **Security Considerations (Development Mode):** Public MQTT broker usage is for testing only; do not publish sensitive payloads. For production, enable TLS and authenticated Kafka, and isolate brokers in VPC networks.
- **Ports (Host → Container):** 9870 (NameNode Web UI), 9092 (Kafka broker), 8080 (Kafka UI), 8083 (Kafka Connect), optionally 5000 (Flask UI) if exposed.

Recommended Optional Tools

- **Postman / curl:** To probe REST endpoints (WebHDFS, Flask).
- **kafkacat / kcat:** Lightweight Kafka CLI for topic debugging.
- **jq:** Command-line JSON processing for inspecting batched files.
- **watch:** Re-run commands periodically (Linux/macOS) e.g., `watch -n 2 docker ps`.

Environment Verification Checklist

Listing 3.1: Quick environment validation

```
# Confirm Docker Engine + Compose plugin
docker version
docker compose version

# Check available resources (Linux/macOS)
free -h          # memory
df -h            # disk space
uname -a         # kernel info

# Verify outbound network (image pulls + MQTT)
ping -c 3 broker.emqx.io
nslookup broker.emqx.io || dig broker.emqx.io

# (Optional) Test Git
git --version

# Confirm no conflicting services occupy project ports
lsof -i :9092
lsof -i :9870
```

Minimum vs Recommended Configuration

Aspect	Minimum	Recommended (Smoother Operation)
vCPUs	2	4+
RAM	4 GB	8 GB
Disk Free	2 GB	10 GB (for data growth)
Docker Version	23.x	24.x or later
Compose Version	v2.17	v2.20+
Network	Stable Wi-Fi	Wired/Ethernet or reliable Wi-Fi

Port Reference

Port	Service	Notes
9870	HDFS NameNode UI	WebHDFS + file browser
9092	Kafka Broker	Broker listener (PLAINTEXT dev mode)
8080	Kafka UI	Topic browsing / offsets
8083	Kafka Connect	Connector management (if enabled)
5000	Flask App	Sensor dashboard (optional)

Common Pre-Run Adjustments

- Increase Docker memory allocation if Kafka or HDFS containers restart unexpectedly.
- Ensure WSL2 backend enabled on Windows for better filesystem performance.
- Disable aggressive antivirus scanning of Docker data directory to reduce I/O latency.
- If using corporate proxy, export HTTP_PROXY/HTTPS_PROXY before image pulls.

Troubleshooting Quick Tips

Listing 3.2: Troubleshooting snippets

```
# Re-pull images if containers fail at start
docker compose pull --quiet

# Remove dangling resources
docker image prune -f
docker volume ls
docker volume prune -f

# Inspect a failing container
docker logs -f namenode
docker inspect kafka | grep -i ipaddress

# DNS resolution test inside a container
docker exec -it python-client bash -c "ping -c 2 kafka"
```

Security / Production Notes

For classroom/demo purposes we accept plaintext protocols. For production:

- Enable TLS + SASL for Kafka.
- Restrict MQTT to authenticated clients with ACLs.
- Use Kerberos or Ranger for fine-grained HDFS access.
- Implement container image scanning and rotate credentials regularly.

4 Files and Repository Layout

A suggested project layout (root directory):

```
docker-compose.yml
scripts/
  m.py    # MQTT -> Kafka
  k.py    # Kafka -> HDFS
flask/
  app.py
  static/
    app.js
  templates/
    index.html
images/
  nit.png
  image1.png
  image2.png
  image3.png
```

Component Purpose

- **docker-compose.yml:** Single orchestration file declaring all services, networks, and volumes for reproducible startup.
- **scripts/m.py:** Bridges MQTT telemetry into Kafka, translating lightweight sensor messages into durable log entries.
- **scripts/k.py:** Consumes Kafka topic **sensors**, batches events, and persists them into date-partitioned HDFS storage.
- **flask/app.py:** Backend service exposing REST/JSON endpoints to serve recent sensor data.
- **flask/static/app.js:** Frontend logic (Chart.js rendering, fetch intervals, UI interactions).
- **flask/templates/index.html:** HTML layout integrating charts, controls, and dynamic data placeholders.
- **images/:** Institutional logo and illustrative figures used in the report and potential dashboard screenshots.

Design Rationale

Clear separation of ingestion (**m.py**), persistence (**k.py**), and visualization (**flask/**) makes the pipeline modular: each layer can evolve independently (e.g., swap MQTT for another protocol, extend consumer for analytics, enhance UI features) without destabilizing the rest. The Compose file centralizes lifecycle management, enabling quick teardown/rebuild for experimentation.

5 Docker Compose Configuration

5.1 Key Settings

- **Kafka Listeners:** PLAINTEXT://kafka:9092 advertised internally; avoids host-IP drift and leverages Docker DNS. External listener (if needed) can be mapped with `KAFKA_ADVERTISED_LISTENERS`.
- **NameNode WebHDFS:** Exposed at host port 9870, enabling REST-style file operations and UI browsing.
- **Script Volume Mount:** `python-client` mounts `./scripts` read/write so editing on host instantly reflects inside container for rapid iteration.
- **Network:** All services join a custom bridge network (e.g., `bigdata`) defined once; network alias equals service name for predictable hostnames.
- **Environment Variables:** Kafka broker ID, log retention hours, and advertised listeners are set via environment: block for declarative configuration.
- **Restart Policies:** Using `restart: unless-stopped` for long-running core services (Kafka, Zookeeper, NameNode) enhances resilience against transient failures.
- **Healthchecks (Recommended):** Add a `healthcheck:` to Kafka (e.g., script invoking `kafka-topics -list`) so dependent services can wait until ready.
- **Volumes:** Bind or named volumes for Kafka data and HDFS storage persist state across container restarts; prevents data loss during iterative development.

5.2 Sample Service Definitions (Excerpt)

Listing 5.1: Snippet from `docker-compose.yml` (illustrative)

```
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:latest
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
    networks: [bigdata]
    restart: unless-stopped

  kafka:
    image: confluentinc/cp-kafka:latest
    depends_on: [zookeeper]
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    ports:
```

```
- "9092:9092"
networks: [bigdata]
restart: unless-stopped

namenode:
  image: bde2020/hadoop-namenode:latest
  environment:
    CLUSTER_NAME: iot_cluster
  ports:
    - "9870:9870"
  networks: [bigdata]
  volumes:
    - hdfs-nn:/hadoop/dfs/name
  restart: unless-stopped

datanode1:
  image: bde2020/hadoop-datanode:latest
  networks: [bigdata]
  volumes:
    - hdfs-dn1:/hadoop/dfs/data
  restart: unless-stopped

python-client:
  image: python:3.10-slim
  working_dir: /scripts
  volumes:
    - ./scripts:/scripts
  networks: [bigdata]
  depends_on: [kafka, namenode]
  command: ["sleep", "infinity"]

networks:
  bigdata:

volumes:
  hdfs-nn:
  hdfs-dn1:
```

5.3 Extended Compose Commands

Listing 5.2: Lifecycle + inspection operations

```
# Validate syntax and interpolated variables
docker compose config

# Start a subset (storage only)
docker compose up -d namenode datanode1

# Scale a stateless worker (if defined)
docker compose up -d --scale python-client=2

# Recreate Kafka after env change (non-cached)
docker compose up -d --force-recreate --no-deps kafka

# Apply resource constraints (edit yaml, then)
docker compose up -d --build
```

```
# View current network and container IPs
docker network inspect bigdata | grep -i "IPv4Address"

# List named volumes in use
docker volume ls

# Stop everything gracefully
docker compose down

# Remove orphaned containers (after service removal)
docker compose up -d --remove-orphans
```

5.4 Health and Logs

Listing 5.3: Monitoring troubleshooting

```
# Stream Kafka logs
docker compose logs -f --tail=100 kafka

# Check container health status (if healthcheck added)
docker inspect --format='{{.State.Health.Status}}' kafka

# Quick disk usage inside NameNode
docker exec -it namenode du -sh /hadoop/dfs/name

# Confirm topic presence
docker exec -it kafka bash -c "kafka-topics --bootstrap-server kafka
:9092 --list"
```

5.5 Resource Tuning (Inline Notes)

- Add `deploy.resources.limits` (CPU, memory) for deterministic performance (Swarm/K8s analog; Compose v2 partly honors).
- For higher throughput, increase Kafka `num.partitions` in topic creation and allocate more RAM.
- To avoid small HDFS files, adjust batch size in `k.py`; Compose ensures persistent volumes for aggregated data.

5.6 Updating Services Safely

Listing 5.4: Rolling update pattern (manual)

```
# Pull latest images
docker compose pull

# Restart critical service (e.g., kafka-ui) without disturbing core
  pipeline
docker compose up -d kafka-ui

# Check status
docker ps --filter "name=kafka-ui"
```

5.7 Cleanup and Maintenance

Listing 5.5: Removing stale artifacts

```
# Remove unused (dangling) images
docker image prune -f

# Prune stopped containers & networks not referenced
docker system prune -f

# Remove volumes ONLY if you want fresh state (data loss!)
docker compose down -v
```

5.8 Why Compose Here

Compose centralizes multi-service orchestration, enabling one-command reproducible startup, consistent networking, and version-controlled infrastructure. It lowers cognitive load versus manual `docker run` sequences and forms a stepping stone toward more advanced schedulers (Kubernetes) while remaining lightweight for a course project.

6 Start / Stop Commands

Start the stack (from project root):

```
docker compose config  
docker compose pull  
docker compose up -d
```

Stop the stack:

```
docker compose down
```

Restart only python client:

```
docker compose up -d python-client
```

7 Verifying Services & Web UIs

- **Container Presence:** `docker ps` should list core services (zookeeper, kafka, kafka-ui, namenode, datanode1, datanode2, python-client). Missing entries suggest some failure.
- **HDFS Web UI:** <http://localhost:9870> — validate NameNode status, browse `/iot/sensors` directory, confirm file creation timestamps.
- **Kafka UI:** <http://localhost:8080> — inspect topic `sensors`, verify partition count, consumer group lag.
- **Basic Connectivity Test:** Inside client: `ping -c 1 kafka` and `curl -f http://namenode:9870` to ensure internal DNS and WebHDFS reachability.
- **Log Inspection:** Use `docker compose logs -f kafka` and `docker compose logs -f namenode` for early error detection (e.g., listener binding, HDFS format issues).

7.1 Kafka Topic Health

Listing 7.1: Kafka topic checks (inside Kafka container)

```
# Enter Kafka container
docker exec -it kafka bash

# List topics
kafka-topics --bootstrap-server kafka:9092 --list

# Describe sensors topic (partitions, replication, configs)
kafka-topics --bootstrap-server kafka:9092 --describe --topic sensors

# Consumer group lag (replace my-group)
kafka-consumer-groups --bootstrap-server kafka:9092 \
  --describe --group my-group

# Produce a quick test message
kafka-console-producer --broker-list kafka:9092 --topic sensors
>{"test":"ping","ts":1699999999}

# Consume latest messages (Ctrl+C to exit)
kafka-console-consumer --bootstrap-server kafka:9092 \
  --topic sensors --from-beginning --max-messages 5
```

Quick Sanity Checklist

- At least one JSON file appears in HDFS after running `k.py`.
- Kafka UI shows increasing message count for `sensors`.
- Consumer group lag approaches zero when `k.py` is active.
- Flask dashboard (if running) updates charts without 404 errors.

8 Python Clients (Source Code Overview)

8.1 m.py (MQTT → Kafka)

- Connects to public MQTT broker and subscribes to Android sensor topics.
- Parses and forwards messages to Kafka topic `sensors`.
- Uses an asynchronous Kafka producer for throughput.
- Adds metadata (ingestion timestamp / device ID) before publishing to Kafka for richer downstream context.
- Implements simple retry logic if broker connection drops (auto reconnect on MQTT callbacks).

8.2 k.py (Kafka → HDFS)

- Polls Kafka topic `sensors`.
- Buffers messages and writes batched JSON files to HDFS under date-partitioned directories.
- Writes either NDJSON or JSON arrays depending on configuration.
- Commits consumer offsets only after successful HDFS write to avoid data loss.
- Rotates batches using size or elapsed time threshold (e.g., 500 records or 30 seconds).

8.3 Running Producers and Consumers

Listing 8.1: Execution inside python-client container

```
# Enter container shell
docker exec -it python-client bash

# Install dependencies (one-time)
pip install hdfs confluent-kafka paho-mqtt

# Optional: verify Python libs
python -c "import confluent_kafka, paho.mqtt.client, hdfs; print('OK')"

# Start HDFS writer (consumer)
python /scripts/k.py

# Start MQTT bridge (producer)
python /scripts/m.py
```

8.3.1 Illustrative Snippet

Listing 8.2: Example transformation helper

```
import pandas as pd
```

```
def normalize(df):  
    """  
    Normalize schema and drop duplicates; coerce timestamps.  
    """  
    df = df.drop_duplicates()  
    if 'timestamp' in df.columns:  
        df['timestamp'] = pd.to_datetime(df['timestamp'], errors=  
            'coerce')  
    return df
```

Minimal Error Handling Pattern (Excerpt)

Listing 8.3: Graceful publish with retry

```
from confluent_kafka import Producer  
import json, time  
  
producer = Producer({"bootstrap.servers": "kafka:9092"})  
  
def send_record(topic, payload):  
    for attempt in range(3):  
        try:  
            producer.produce(topic, json.dumps(payload))  
            producer.poll(0)  
            return True  
        except BufferError:  
            producer.poll(0.5) # backoff  
    return False
```

9 Frontend Web UI – IoT Sensor Visualizer

9.1 Summary

A Flask app reads recent partitioned HDFS files and exposes JSON/REST endpoints consumed by a Chart.js frontend to render acceleration and gyroscope charts. The UI supports:

- Manual refresh and auto-refresh (JavaScript interval polling).
- Selecting number of samples (query parameter e.g., `?limit=200`).
- Viewing timestamped sensor traces for x, y, z axes.
- Basic status banner showing last file ingestion time.

9.2 Endpoint Sketch

- GET `/api/latest?limit=N`: Returns most recent N sensor readings (merged across recent HDFS batch files).
- GET `/api/health`: Simple JSON (`"status":"ok", "lastwrite": "..."`) *for uptime check*.
- GET `/`: Serves `index.html` with embedded Chart.js logic.

9.3 HDFS Integration Notes

Use `InsecureClient('http://namenode:9870', user='root')` from within the Docker network so DataNode redirects resolve to container hostnames. Run the Flask app inside the compose network (`python-client`) for correct connectivity. Cache a small in-memory slice (e.g., last 500 records) to reduce repeated HDFS reads on rapid auto-refresh.

9.4 Launching the UI

Listing 9.1: Flask UI run (inside python-client)

```
# Set Flask app (if needed)
export FLASK_APP=/flask/app.py

# Install Flask if not present
pip install flask

# Run development server on 0.0.0.0:5000
flask run --host=0.0.0.0 --port=5000
```

Minimal Flask Route Example

Listing 9.2: Simplified Flask app fragment

```
from flask import Flask, jsonify, request
from hdfs import InsecureClient

app = Flask(__name__)
client = InsecureClient('http://namenode:9870', user='root')

CACHE = [] # last cached records (dicts)

@app.route('/api/latest')
def latest():
    limit = int(request.args.get('limit', 100))
    # If cache empty or stale, read newest file (simplified placeholder)
    ):
    if not CACHE:
        # list directory, pick newest file, read & parse JSON
        # records = ...
        # CACHE.extend(records)
        pass
    return jsonify(CACHE[-limit:])

@app.route('/api/health')
def health():
    return jsonify({"status": "ok", "cached_records": len(CACHE)})

# index.html served by a template with Chart.js fetching /api/latest
```

Brief Design Rationale

A thin Flask layer keeps server logic simple while allowing flexible frontend refresh strategies. Partitioned HDFS storage + lightweight caching yields acceptable latency for classroom-scale datasets without introducing heavier streaming frameworks.

Prepared from the project notes provided.
(Source document used: uploaded project report.)